

Total No. of Questions : 8]

PD4249

SEAT No. :

[Total No. of Pages : 2

[6403]-44

T.E. (Computer Engg.)

WEB TECHNOLOGY

(2019 Pattern) (Semester - VI) (310252)

Time : 2½ Hours]

[Max. Marks : 70

Instructions to the candidates:

- 1) Answer Q.1 or Q.2, Q.3 or Q.4, Q5 or Q6, Q7 or Q8.
- 2) Neat diagrams must be drawn wherever necessary.
- 3) Figures to the right side indicate full marks.
- 4) Assume suitable data if necessary.

Q1) a) Compare doGet and doPost methods in Servlet. [8]

b) What is Servlet? Explain the life cycle of servlet. Illustrate with example. [9]

OR

Q2) a) What is session? List the session tracking techniques. How cookies are used to track a session. [9]

b) Distinguish between external and internal DTDs. [8]

Q3) a) List and elaborate any five JSP implicit objects with example. [8]

b) Explain the Struts architecture with neat diagram and also explain the benefits of Struts. [9]

OR

Q4) a) Write a JSP program to demonstrate use of page directive, Scriptlet, Expression and Comment. [9]

b) Identify and justify the benefits of Web Services. [8]

Q5) a) Explain different types of arrays in PHP with example. [9]

b) How does this function work in PHP? Explain with example. [9]

OR

P.T.O.

PDF BY VL



Telegram Channel

[Click here to join](#)



Instagram Page

[Click here to join](#)



WhatsApp Channel

[Click here to join](#)



★ WHY JOIN ?

- 🏆 Trusted by 80,000+ Engineering Students
- 🔔 Stay Updated with latest university news
- 🚀 Make your engineering journey easy with us

Best Regards, **Vishwajeet Londhe**

Q6) a) Write php code to connect MYSQL database to display records from the table. [9]

b) Draw and explain.Net framework. [9]

Q7) a) Explain types of Arrays in Ruby? Explain the array operation in Ruby. [9]

b) What are the positive aspects of Rails, explain with example. [9]

OR

Q8) a) What is a module? Explain the difference between classes and modules? [9]

b) What is EJB? Draw and explain main components of EJB architecture. [9]



Solutions:

Q1 a) doGet() and doPost() are methods of the HttpServlet class. They handle HTTP GET and HTTP POST requests sent by the client.

Ans:

Comparison (Write any 8 for full marks)

doGet()	doPost()
Handles HTTP GET requests	Handles HTTP POST requests
Sends data through URL	Sends data in request body
Data is visible in URL	Data is hidden from URL
Not secure for sensitive data	More secure for sensitive data
Limited data can be sent (because URL length is limited)	Large amount of data can be sent
Request can be bookmarked or cached	Cannot be bookmarked or cached
Mostly used for fetching / reading data	Mostly used for submitting / updating data
Faster because no body processing	Slightly slower due to request body handling
Can be tested directly by typing URL	Cannot be tested by URL alone; form required
Idempotent (same request → same result)	Not necessarily idempotent

Q1 (b) What is Servlet? Explain the life cycle of servlet. Illustrate with example. [9 Marks]

Ans:

A **Servlet** is a Java program that runs on a web server, receives client requests, processes them, and sends responses (usually HTML).

Servlets are used to create **dynamic web pages**.

What is a Servlet?

1. Servlet is **server-side Java component** that handles web requests.
2. It extends **javax.servlet.http.HttpServlet**.
3. Runs inside a **Servlet Container** such as Tomcat.
4. Used to generate **dynamic content** (HTML, JSON, XML).
5. More efficient and secure than CGI programs.

Servlet Life Cycle

Servlet lifecycle has **5 important stages**:

1. Loading the Servlet

- Servlet container loads the servlet class into memory.
- Happens at first request or when server starts.

2. Instantiation

- Container creates **one single object** of the servlet.
- Shared across multiple user requests.

3. Initialization – init()

- Called **only once** in servlet lifetime.
- Used to initialize:
 - Database connections
 - Configuration parameters
 - Resources

```
public void init() throws ServletException {  
    // initialization code  
}
```

4. Request Handling – service()

- Called **for every client request**.
- Determines request type:
 - Calls doGet() for GET
 - Calls doPost() for POST

```
public void service(HttpServletRequest req, HttpServletResponse res) {  
    // process request  
}
```

5. Destruction – destroy()

- Called when servlet is removed from service.
- Used to release resources.

```
public void destroy() {  
    // cleanup activities  
}
```

Servlet Life Cycle Diagram Servlet Example

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
  
public class HelloServlet extends HttpServlet {  
  
    public void init() {  
        System.out.println("Servlet Initialized");  
    }  
  
    public void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws IOException {  
  
        res.setContentType("text/html");  
        PrintWriter out = res.getWriter();  
        out.println("<h2>Hello World from Servlet</h2>");  
    }  
  
    public void destroy() {  
        System.out.println("Servlet Destroyed");  
    }  
}
```

Load class

Create object

Call init()

For every request → service() → doGet()/doPost()

Shutdown → destroy()

OR

Q2) a) What is session? List the session tracking techniques. How cookies are used to track a session. [9]

Ans:

A **session** is a way to store user-specific information across multiple requests in a web application. Since HTTP is **stateless**, sessions help the server remember the same user.

a) What is session?

Point-to-point:

1. A session is a **temporary storage** created for each user.
2. It stores user data (login info, cart items, preferences).
3. It helps identify the same user during multiple page requests.
4. Sessions exist between **login** → **logout** or until timeout.
5. They are stored on the server and identified by a session ID.

List the session tracking techniques

Point-to-point:

1. **Cookies**
2. **URL Rewriting**
3. **Hidden Form Fields**
4. **HTTPS Session Tracking (SSL)**
5. **Session API (HttpSession in Java)**

How cookies are used to track a session?

Point-to-point explanation:

1. When a user first visits, the server creates a **session ID**.
2. This ID is stored inside a **cookie** in the user's browser.
3. On every new request, the browser automatically sends the cookie back.
4. The server reads the cookie → identifies the session ID.
5. Server maps the session ID to stored data (user info, cart, etc.).
6. This way, cookies help maintain **continuous user identity**.
7. Even if the user moves between pages, the session stays active.

b) Distinguish between external and internal DTDs.[8]

Ans:

A **DTD (Document Type Definition)** defines the structure and rules of an XML document.

DTD can be written **inside** the XML file (internal) or stored **outside** in a separate file (external).

Difference between External DTD and Internal DTD

Point-to-point table format:

Internal DTD	External DTD
1. Written inside the XML file.	1. Stored in a separate .dtd file .
2. Defined using <!DOCTYPE> inside the same document.	2. Linked using SYSTEM or PUBLIC keyword.
3. Easy to create for small XML files.	3. Reusable across multiple XML files.
4. Good for simple structures.	4. Good for large or shared projects.
5. Not ideal for big definitions.	5. Reduces duplication for large XML systems.
6. Faster loading (no external file fetching).	6. Slight delay due to external file loading.
7. Modifications affect only one XML file.	7. Modifications reflect in all connected XML files.

Q3 a) List and elaborate any five JSP implicit objects with example.[8]

Ans:

JSP implicit objects are **predefined objects** created by the JSP container. They can be used directly in the JSP page **without creating them manually**.

Five JSP Implicit Objects (Any 5)

1 request

- Type: HttpServletRequest
- Used to get **input values**, request headers, parameters, client info.

Example

```
<%= request.getParameter("name") %>
```

2 response

- Type: HttpServletResponse
- Used to control **HTTP response**, redirect pages and set content type.

Example

```
<%  
    response.sendRedirect("home.jsp");  
%>
```

3 out

- Type: JspWriter
- Used to print output inside JSP.
- Similar to System.out.println.

Example

```
<%  
    out.println("Welcome User!");  
%>
```

4 session

- Type: HttpSession
- Used to store user-specific data across multiple requests.

Example

```
<%  
    session.setAttribute("username", "Vijay");  
%>
```

5 application

- Type: ServletContext
- Used to share data across **entire application** (all users, all pages).

Example

```
<%  
    application.setAttribute("counter", 1);  
%>
```

6 config (Extra if needed)

- Type: ServletConfig
- Stores JSP/Servlet configuration parameters.

Example

```
<%= config.getInitParameter("adminEmail") %>
```

7 exception (Only in error pages)

Used to display exception details.

Q3 b) Explain Struts architecture with neat diagram and benefits of Struts.[9]

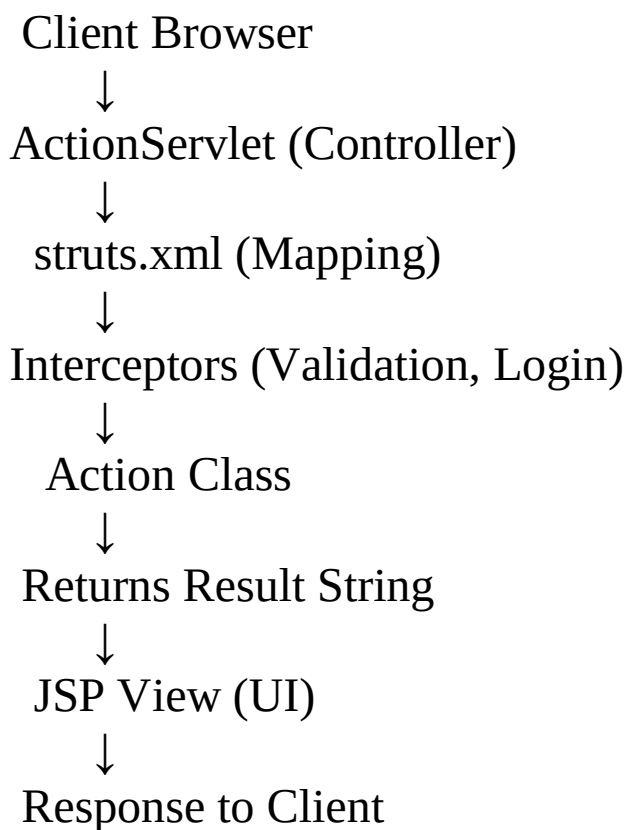
Ans:

Struts is an **MVC-based Java web framework** used to build large-scale applications.

It uses **ActionServlet, Action classes, struts.xml, interceptors, and JSP views.**

1. Struts Architecture

Below is a clean text-diagram you can draw in exam:



Components Explained

1 ActionServlet (Front Controller)

- Receives all requests.
- Looks into struts.xml for mapping.
- Sends request to the correct Action class.

2 struts.xml

- Central configuration file.
- Maps URL → Action class → Result page.

Example:

```
<action name="login" class="com.demo.LoginAction">
  <result name="success">home.jsp</result>
  <result name="error">login.jsp</result>
</action>
```

3 Interceptors

- Pre-processing & post-processing
- Examples: validation, authentication, logging

4 Action Class

- Java class that contains business logic.

Example:

```
public class LoginAction extends ActionSupport {
  public String execute() {
    if(user.equals("admin")) return "success";
    else return "error";
  }
}
```

5 JSP View

- Displays result to user
- Uses Struts tag libraries

2. Benefits of Struts

1. Follows MVC architecture

Keeps UI (JSP), logic (Action), and routing (XML) separate.

2. Faster development

Many built-in features like validation, form handling.

3. Centralized configuration

All mappings handled in one file (struts.xml).

4. Easy integration

Works well with Hibernate, Spring, JSP, JDBC.

5. Reusable Interceptors

Used for logging, validation, security.

6. Scalable

Used for large enterprise-grade applications.

OR — Q4 a) JSP program for page directive, scriptlet, expression & comment.[9]

Ans:

Demonstrates:

- Page directive
- Scriptlet
- Expression
- Comment

```
<%@ page language="java" contentType="text/html" %> <!-- Page Directive -->

<html>

<body>

<%-- This is JSP Comment --%>

<%

    // Scriptlet

    int x = 10, y = 20;

    int sum = x + y;

    String name = "Vijay";

%>

<p>Using JSP Expression:</p>

Name: <%= name %> <br>

Sum: <%= sum %>

</body>

</html>
```

Q4 b) Identify and justify the benefits of Web Services.[8]

Ans:

Benefits of Web Services

1 Platform Independent

- Uses XML/JSON so Java, Python, .NET can communicate.

2 Language Independent

- Client and server may be written in different languages.

3 Loosely Coupled

- Changes in server code do not affect clients.

4 Interoperability

- Web services work across OS, devices & networks.

5 Reusable

- Same service can be used in mobile app, web app, desktop app.

6 Uses Standard Protocols

- HTTP, SOAP, XML, WSDL → universal support.

7 Supports Distributed Systems

- Ideal for cloud applications & enterprise systems.

8 Secure and Reliable (SOAP-based services)

- SOAP supports encryption, digital signatures, WS-security.

Q5) a) Explain different types of arrays in PHP with example. [9]

Ans:

In PHP, arrays are used to store multiple values in a single variable. PHP provides **three main types of arrays** depending on how data is indexed.

1) Indexed Array

- Stores values with **numeric index (0,1,2...)**
- Default index is automatically assigned.

Example

```
<?php
$cities = ["Pune", "Mumbai", "Nashik"];
echo $cities[1]; // Output: Mumbai
?>
```

2) Associative Array

- Uses **user-defined keys** instead of numbers.
- Keys are strings.

Example

```
<?php
$student = [
    "name" => "Vijay",
    "age" => 21,
    "city" => "Pune"
];
echo $student["name"];
?>
```

3) Multidimensional Array

- Array inside another array.
- Used for storing table-like data.

Example

```
<?php
$employees = [
    ["Rohan", 25000],
    ["Sneha", 30000],
    ["Amit", 27000]
];
echo $employees[1][0]; // Output: Sneha
?>
```

4) Associative Multidimensional Array

- Combination of associative + multidimensional array.

Example

```
<?php
$products = [
    "Laptop" => ["price" => 55000, "brand" => "HP"],
    "Mobile" => ["price" => 20000, "brand" => "Samsung"]
];
echo $products["Laptop"]["brand"];
?>
```

Q5) b) How does `this` function work in PHP? Explain with example. [9]

Ans:

In PHP, `$this` is used **inside a class** to refer to the **current object**. It allows accessing **properties and methods** of the same object.

How it Works – Pointwise

1. `$this` can be used **only inside a class**.
2. It refers to the **object that calls the method**.
3. It is used to access:
 - Current object's properties
 - Current object's methods
4. Without `$this`, we cannot access non-static members inside class.
5. `$this` holds memory reference of the object at runtime.

Example Code

```
<?php
class Student {
    public $name;

    function setName($n) {
        $this->name = $n;    // using $this to access class property
    }

    function getName() {
        return $this->name; // using $this again
    }
}

$obj = new Student();
$obj->setName("Vijay");
echo $obj->getName();
?>
```

Output

Vijay

Explanation

- `$this->name` means **name property of the current object \$obj**.
- `$this->setName()` refers to the method belonging to the same object.

OR-Q6) a) Write PHP code to connect MySQL database to display records from the table. [9]

Ans:

PHP uses **MySQLi** or **PDO** to connect MySQL database.
We write:

- Connection code
- Query
- Fetch data
- Display data

PHP Code

```
<?php
// Step 1: Connect to MySQL
$conn = mysqli_connect("localhost", "root", "", "college");

// Step 2: Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Step 3: Run SQL query
$sql = "SELECT id, name, course FROM students";
$result = mysqli_query($conn, $sql);

// Step 4: Display records
while ($row = mysqli_fetch_assoc($result)) {
    echo $row["id"] . " - " . $row["name"] . " - " . $row["course"] . "<br>";
}

// Step 5: Close connection
mysqli_close($conn);
?>
```

Q6) b) Draw and explain .NET Framework. [9]

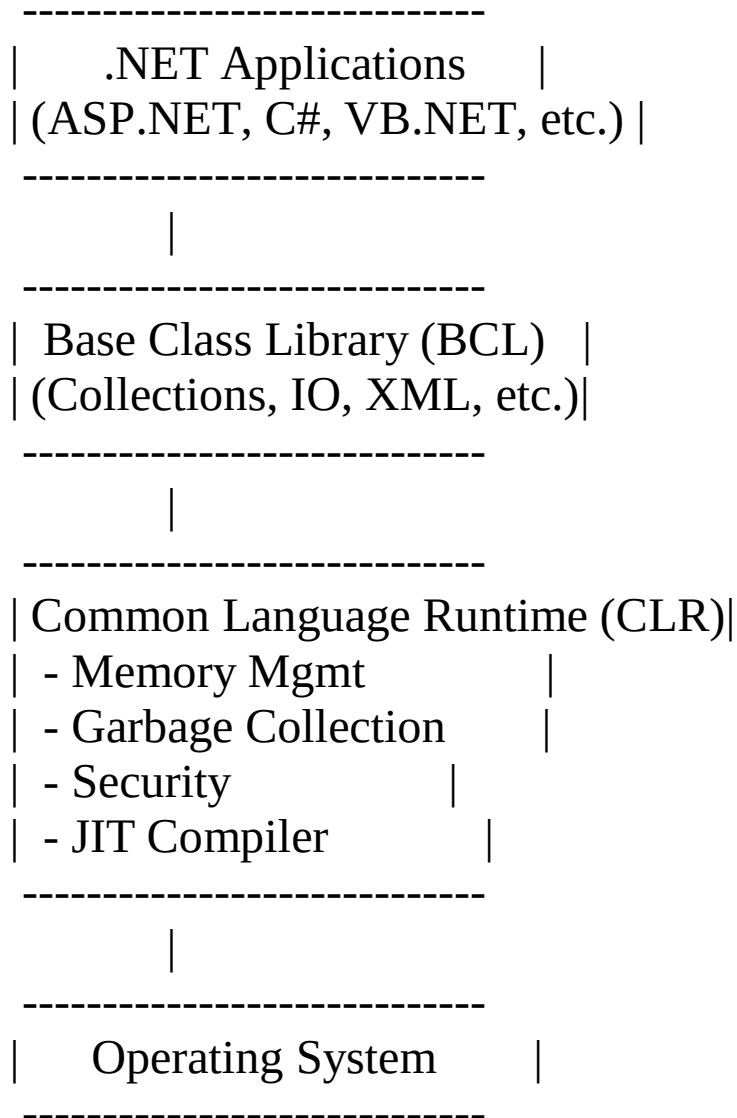
Ans:

.NET Framework is a **Microsoft platform** used to develop:

- Web apps
- Desktop apps
- Mobile apps

It provides **runtime environment + class libraries**.

Diagram (Text-Based for exam)



Explanation – Pointwise

1) .NET Applications

- Web apps (ASP.NET)
- Desktop apps
- Services

2) Base Class Library (BCL)

- Ready-made classes
- Supports strings, files, networking, XML, etc.

3) Common Language Runtime (CLR)

- Main execution engine
- Duties:
 - Converts IL → Machine code (JIT)
 - Memory management
 - Garbage collection
 - Security
 - Exception handling

4) OS Layer

- CLR depends on OS for hardware access.

Q7 a) Explain types of Arrays in Ruby. Explain array operations in Ruby.[9]

Ans:

In Ruby, an **Array** is an ordered list of elements. Ruby arrays can store **mixed data types**, such as Integer, String, Float, etc.

1) Types of Arrays in Ruby

i) Numeric Array

Stores only numbers.

```
nums = [10, 20, 30]
```

ii) String Array

Stores text values.

```
names = ["Ram", "Vijay", "Neha"]
```

iii) Mixed Array

Stores multiple data types.

```
mix = [10, "Ruby", 4.5, true]
```

iv) Array of Arrays (2-D Array)

```
matrix = [[1,2],[3,4]]
```

v) Empty Array

```
empty = []
```

2) Array Operations in Ruby

i) Accessing Elements

```
arr = [10, 20, 30]
```

```
arr[0] # 10
```

```
arr[2] # 30
```

ii) Adding Elements

```
arr << 40      # append
```

```
arr.push(50)   # push
```

```
arr.unshift(5) # add at beginning
```

iii) Removing Elements

```
arr.pop        # remove last
```

```
arr.shift      # remove first
```

```
arr.delete(20) # deletes 20 if exists
```

iv) Updating Elements

```
arr[1] = 99
```

v) Length of Array

```
arr.length
```

```
arr.size
```

vi) Iteration

```
arr.each do |x|  
  puts x  
end
```

vii) Searching

```
arr.include?(30)  
arr.index(50)
```

viii) Sorting

```
arr.sort  
arr.reverse
```

ix) Joining Array to String

```
["a","b","c"].join("-") # "a-b-c"
```

x) Splitting String to Array

```
"ruby,rails".split(",")
```

Q7 b) What are the positive aspects of Rails? Explain with example.[9]

Ans:

Rails is a **web application framework** for Ruby that supports rapid development using **MVC, Convention over Configuration, and DRY principles.**

Positive Aspects of Rails (With Examples)

1) MVC Architecture

1) MVC Architecture

Rails divides application into:

- **Model** → Database
- **View** → User Interface
- **Controller** → Logic

Helps in clean, maintainable apps.

2) Convention Over Configuration (CoC)

Rails reduces configuration by following naming rules.

Example:

Model Student → table name automatically **students**.

3) DRY Principle (Don't Repeat Yourself)

Code reusability improves development speed.

Example:

Common login code reused via helpers.

4) ActiveRecord (Built-in ORM)

No need to write SQL.

Example:

```
@student = Student.new(name: "Vijay")  
@student.save
```

5) Scaffolding

Auto-generates:

- Model
- Controller
- Views
- Migration

Command:

```
rails generate scaffold Student name:string age:integer
```

6) Built-in Security

Rails protects against:

- SQL Injection
- CSRF
- XSS

7) Huge Community & Gems

Thousands of plugins:

- Devise (Authentication)
- Pundit (Authorization)

8) Rapid Development

Simple form example:

```
<%= form_with model: @student do |f| %>
  <%= f.text_field :name %>
  <%= f.submit "Save" %>
<% end %>
```

OR -Q8 a) What is a module? Explain the difference between classes and modules.[9]

Ans:

A **module** in Ruby is a collection of **methods, constants, and mixins**, but **cannot create objects**.

Modules are used for:

- Namespaces
- Method sharing
- Mixins

Definition of Module

A module groups reusable code.

Example:

```
module Greetings
  def say_hello
    puts "Hello"
  end
end
```

Including a Module

```
class Student
  include Greetings
end

s = Student.new
s.say_hello
```

Difference Between Classes and Modules

Feature	Class	Module
Object creation	✔Objects can be created	✘Objects cannot be created
Inheritance	✔Supports inheritance	✘No inheritance
Multiple inheritance	✘Not allowed	✔Achieved via mixins
Purpose	Define objects	Group methods & constants
Example	class Student	module MathTools

Example

```
module Calc
  def add(a,b)
    a+b
  end
end
```

```
class Test
  include Calc
end
```

```
puts Test.new.add(2,3)
```

Q8 b) What is EJB? Draw and explain main components of EJB architecture.[9]

Ans:

EJB (Enterprise JavaBeans) is a **server-side component model** used to build **large, secure, scalable enterprise applications**.

Explanation of Components

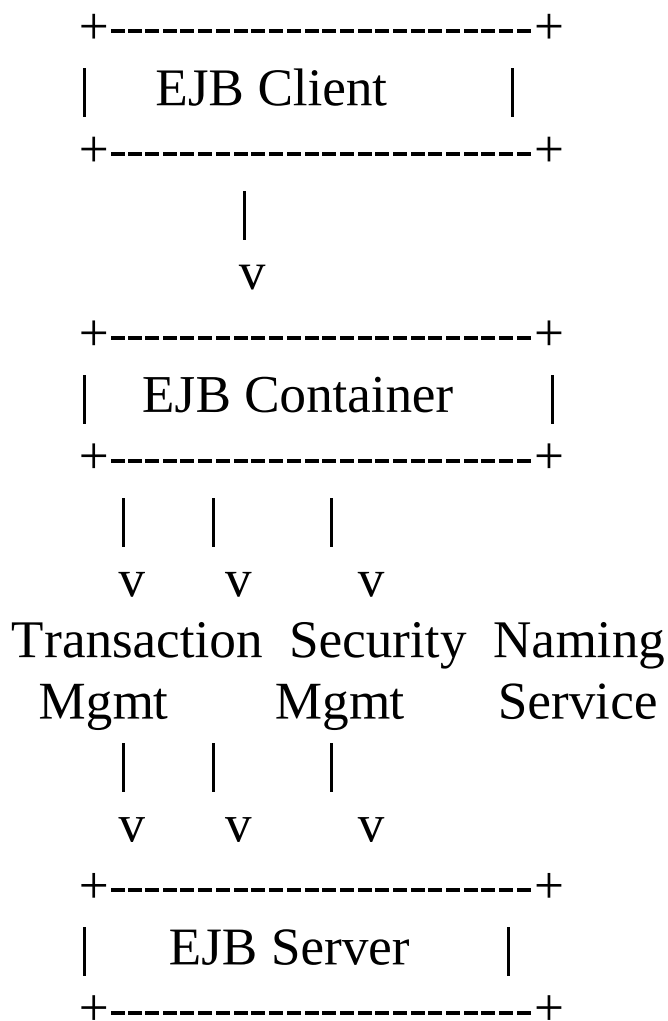
1) EJB Client

Requests services from EJB.

Types:

- Web browser
- Java application
- Mobile app

Components of EJB Architecture (Diagram + Explanation)



2) EJB Container

The most important component; manages:

- Lifecycle
- Transactions
- Security
- Concurrency

Container handles all system-level work.

3) EJB Server

Provides environment where container runs.

It handles:

- Deployment
- Resource management
- Connection pools

4) System Services

a) Transaction Management

Ensures ACID properties.

b) Security

Role-based security.

c) Naming / JNDI

Lookup service for EJB components.

PDF BY VL



Telegram Channel

[Click here to join](#)



Instagram Page

[Click here to join](#)



WhatsApp Channel

[Click here to join](#)



WHY JOIN ?

- 🏆 Trusted by 80,000+ Engineering Students
- 🔔 Stay Updated with latest university news
- 🚀 Make your engineering journey easy with us

Best Regards, **Vishwajeet Londhe**